FIVE-DRONE SWARM

Leader–Follower Formation, Dynamic Area Survey,
Task Reallocation and Dynamic Leader Election

Executive summary

The project implements a five-vehicle multi-drone coordination system in simulation. It combines formation flight, independent ROS 2 processes communicating through DDS, area-survey task allocation, progress reporting, programmed-delay task reassignment, and a later dynamic leader-election extension.

The original variants use drone0 as the fixed leader. The dynamic variant removes that hard-coded role: all five drones run the same dynamic_swarm.py program and maintain leadership state through periodic heartbeats. The operator can select a leader manually or return to automatic selection.

1. Problem statement and objectives

The project aims to develop a five-drone swarm system using ROS 2 Humble, AeroStack2, and Gazebo simulation, with vehicles identified as drone0 through drone4. The system supports autonomous takeoff and landing, line and V-shaped formations, and high-level drone control through the AeroStack2 DroneInterface, along with direct position-feedback control using Gazebo velocity topics. It enables communication between the leader and follower drones through ROS 2 topics and DDS, distributes survey waypoints among individual followers, and monitors their progress. If a drone experiences a programmed delay, its unfinished waypoints can be reassigned to a helper drone. The project also extends the swarm with dynamic leader election, manual leader selection, and failure-recovery mechanisms, allowing the drones to adapt their leadership roles during operation.

2. Software stack and environment

Component	Version / role
Operating system	Ubuntu 22.04
Middleware	ROS 2 Humble
Aerial robotics framework	AeroStack2
Simulator	Gazebo / Ignition Fortress
Python interface	rclpy
Messages	geometry_msgs, std_msgs
Transport	DDS / Fast DDS
Process management	Bash and tmux
Languages	Python 3 and Bash

3. Architecture

Layer	Responsibility	Implementation
Simulation	Vehicle models, physics and simulator topics	Gazebo / Ignition
Vehicle abstraction	Arm, offboard, takeoff, GoTo and land	AeroStack2 DroneInterface
Motion control	Pose feedback, target calculation and velocity generation	direct_formation.py and DDS programs
Coordination	Leader commands and leader-pose distribution	leader_follower_dds.py
Task execution	Survey tasks, waypoint tracking and progress	leader_tasks.py
Role management	Runtime election, manual override and failover	election.py, dynamic_swarm.py, swarm_cli.py

4. Five-drone roles and task areas

Drone	Original role	Initial responsibility
drone0	Leader	Formation reference, commands, mission supervision
drone1	Follower 1	Survey area A
drone2	Follower 2	Survey area B
drone3	Follower 3	Survey area C
drone4	Follower / helper	Available for reassigned survey work

Area	Initial owner	Waypoints (x, y), metres
A	drone1	(-2.5,-2.0) → (-2.5,2.0) → (-1.0,2.0) → (-1.0,-2.0)
B	drone2	(-0.5,-2.0) → (-0.5,2.0) → (0.5,2.0) → (0.5,-2.0)
C	drone3	(1.0,-2.0) → (1.0,2.0) → (2.5,2.0) → (2.5,-2.0)
Helper	drone4	Available for reassignment

In the documented delay demonstration, drone1 completes A1 and A2, reports a delay with A3 and A4 unfinished, and the leader transfers those two waypoints to drone4 while commanding drone1 to HOLD. The helper completes A3 and A4.

5. Formation control

Formation geometry is expressed as fixed horizontal offsets from the current leader. In the dynamic variant, followers sorted by ID fill the available slots behind whichever drone currently leads.

Slot	Line offset (x,y) m	V / arrow offset (x,y) m
Leader	(0,0)	(0,0)
1	(-1.5,0)	(-1.5,+1.5)
2	(-3.0,0)	(-1.5,-1.5)
3	(-4.5,0)	(-3.0,+3.0)
4	(-6.0,0)	(-3.0,-3.0)

The proportional controller calculates position error as target minus current position. Horizontal velocity commands are proportional to x/y error; the vertical term uses twice the proportional gain. Commands are clipped to configured limits.

Parameter	Documented value
Target altitude / ground reference	1.05 m / 0.06 m
Proportional gain Kp	0.8
Horizontal velocity limit	0.6 m/s
Climb velocity limit	0.5 m/s
Descent velocity limit	0.4 m/s
Position tolerance	0.15 m (0.2 m in task program)

6. ROS 2 communication interfaces

Topic	Message type	Purpose
/formation/command	std_msgs/String	Formation and flight commands
/formation/leader_pose	geometry_msgs/PoseStamped	Leader pose reference
/formation/task/droneN	std_msgs/String (JSON)	Individual follower task
/formation/status	std_msgs/String (JSON)	Follower progress and task status
/swarm/heartbeat	std_msgs/String (JSON)	Dynamic heartbeat and leader claim
/swarm/cmd	std_msgs/String	Operator commands
/droneN/ground_truth/pose	geometry_msgs/PoseStamped	Ground-truth position feedback
/gz/droneN/cmd_vel	geometry_msgs/Twist	Gazebo velocity command

7. Dynamic leader election

All five drones run the same dynamic_swarm.py process. The selection criterion is chosen at launch: battery (default), ID or centroid. An operator can pin a manually selected leader using leader N and return to automatic selection using auto.

Mode	Selection rule	Planned handover
battery	Highest simulated battery; ties go to lowest ID	Challenger must lead by at least 5 percentage points
id	Lowest alive drone ID	Does not pre-empt a live leader
centroid	Drone closest to swarm centroid	Challenger must be at least 1 m closer
manual	Operator-selected drone	Operator changes it; failure failover remains active

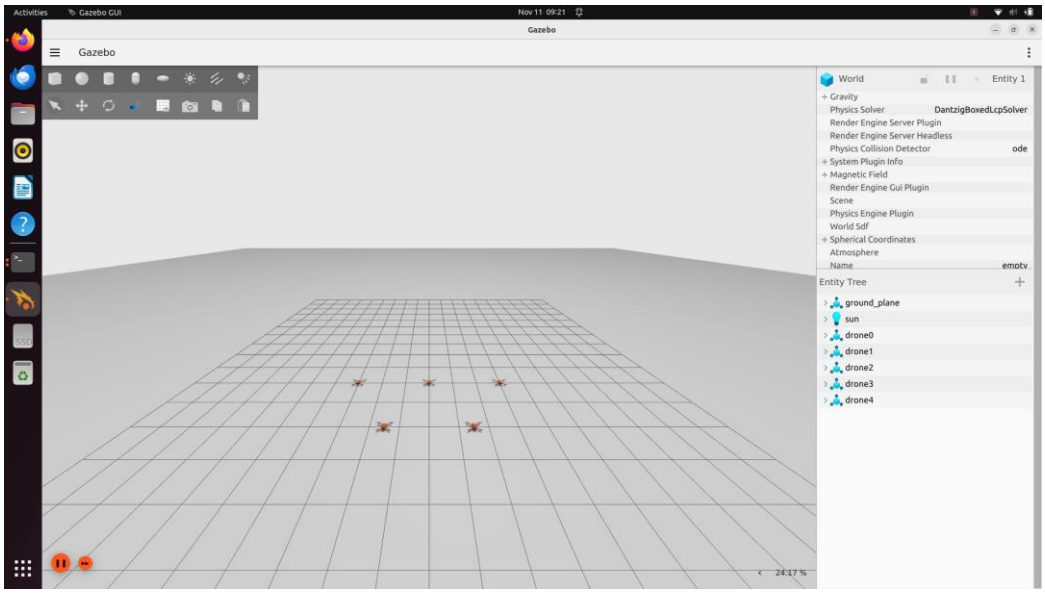
Protocol details:

- Heartbeat rate: 10 Hz; heartbeat contains ID, battery, position, mode, current claim (term and leader) and manual-pin state.
- A drone is considered alive when a heartbeat has been received within 1.5 seconds.
- Claim ordering: higher term wins; for equal terms, lower leader ID wins.
- Initial election: after 3 seconds without a claim, each drone computes the best alive drone and claims term 1.
- Leader failure: followers claim the next term with the best alive candidate; conflicting claims resolve by the claim ordering.
- Planned handover: only the current leader initiates it, using hysteresis to prevent threshold races and flapping.
- Manual handover: leader N is sent to all drones; a dead target is refused. Automatic handovers are pinned off until auto is requested.
- Rejoining drones adopt the current swarm claim instead of forcing a new election.
- Each drone flies at a separate level: 1.05 m + 0.3 m × drone ID.

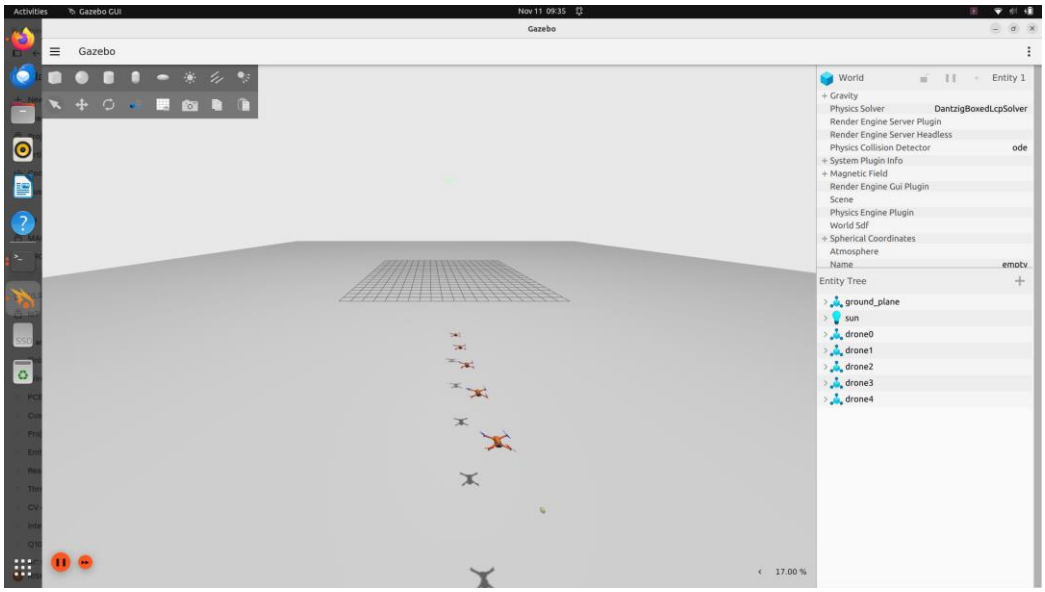
8. Operator console commands

Command	Effect
T / L	Take off / land all drones
1 / 2	LINE / ARROW (V) formation
g dx dy	Move current leader; followers follow
leader N	Assign drone N as manual leader and pin selection
auto	Resume criterion-based automatic selection
fail N / recover N	Simulate a drone going silent / rejoining
batt N PERCENT	Set simulated battery percentage
status	Show leader, battery, mode, term, consensus and selection mode
q	Quit

9. Selected simulation evidence

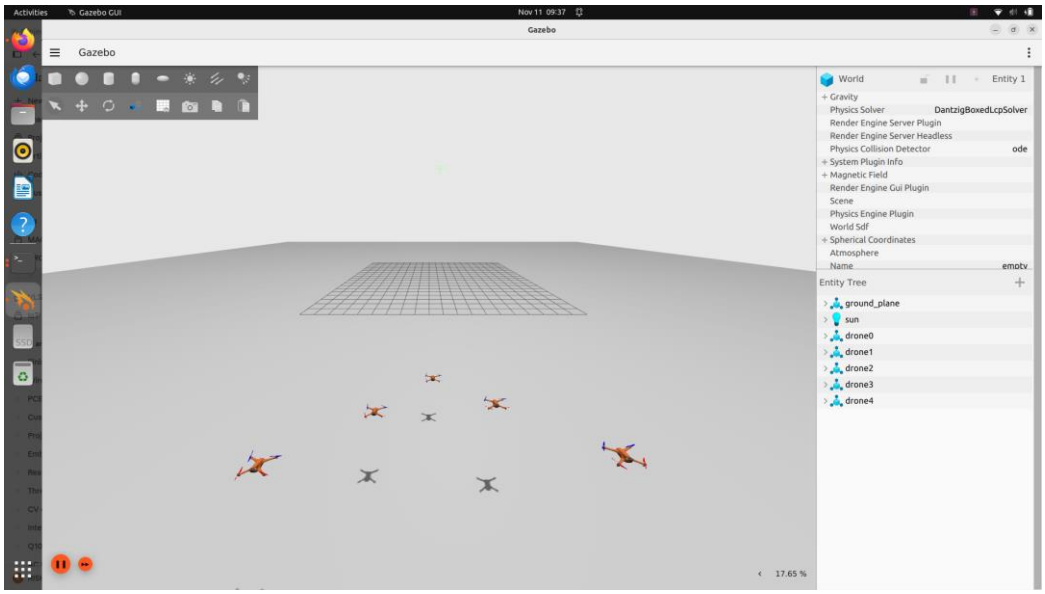


Five drone models spawned in Gazebo.



Dynamic line formation behind the elected leader.

FIVE-DRONE SWARM



Dynamic V / arrow formation behind the elected leader.

10. Operating procedure

10.1 Environment setup

```
source /opt/ros/humble/setup.bash
source ~/aerostack2_ws/install/setup.bash
export FASTRTPS_DEFAULT_PROFILES_FILE="$HOME/project_gazebo/config/fastdds_udp_only.xml"
export FASTDDS_DEFAULT_PROFILES_FILE="$HOME/project_gazebo/config/fastdds_udp_only.xml"
export ROS2CLI_DISABLE_DAEMON=1
cd ~/project_gazebo
```

10.2 Start simulation

```
./launch_as2_5drone.bash -m
tmux list-sessions
ros2 node list
ros2 topic list
```

Expected vehicle sessions/namespaces: drone0, drone1, drone2, drone3 and drone4. Confirm the simulation is running and ground-truth pose topics are available before starting a controller.

10.3 Dynamic swarm demonstration

```
python3 test_election.py
./run_dynamic_swarm.sh battery

# In the swarm console:
batt 0 100
batt 1 95
batt 2 90
batt 3 85
batt 4 80
status
T
1
2
batt 0 60
leader 3
status
fail 3
recover 3
auto
L
q
```

Note: the fail/recover commands are supported by the console and verified offline; the supplied evidence does not show a live Gazebo failover recording.

10.4 Other program variants

Variant	Command
AeroStack2 command demo	python3 leader_follower_command_demo.py
Direct formation controller	python3 direct_formation.py
DDS leader–follower formation	./run_leader_follower.sh
Task allocation and reassignment	./run_leader_tasks.sh
Dynamic leader election	./run_dynamic_swarm.sh battery

Run only one control variant at a time. Do not run multiple controllers against the same drones concurrently.

10.5 Diagnostics and shutdown

```
ros2 topic list | grep -E "ground_truth|cmd_vel"
ros2 topic echo /drone0/ground_truth/pose --once
cat /tmp/swarm0.log
tail -n0 -F /tmp/swarm*.log | grep --line-buffered -E "LEADER|MANUAL|AUTO|FAILURE|recovered"

# Stop dynamic processes
pkill -f dynamic_swarm.py
pkill -f swarm_cli.py
./stop.bash

# Full reset if stale Gazebo/tmux sessions block a new run
tmux kill-server
pkill -f "ign gazebo"
```

COMPLETE PROGRAM SOURCE CODE

Appendix 1: leader_follower_command_demo.py

```
#!/usr/bin/env python3
import sys
import time

import rclpy
from as2_python_api.drone_interface import DroneInterface

# CONFIGURATION
DRONE_NAMES = [
    "drone0",
    "drone1",
    "drone2",
    "drone3",
    "drone4",
]

LEADER = "drone0"

TAKEOFF_HEIGHT = 1.0
TAKEOFF_SPEED = 0.5
MISSION_SPEED = 0.6
LAND_SPEED = 0.4
FRAME_ID = "earth"
STABILIZATION_TIME = 3.0
FORMATION_SPACING = 1.5

# Leader's nominal formation reference position.
LEADER_X = 0.0
LEADER_Y = 0.0
FLIGHT_HEIGHT = TAKEOFF_HEIGHT

# FORMATION OFFSETS
# Positions are relative to the leader's nominal
# reference position, using the earth frame.

# LINE: followers form a line behind the leader.
LINE_OFFSETS = {
    "drone1": (-FORMATION_SPACING, 0.0),
    "drone2": (-2 * FORMATION_SPACING, 0.0),
    "drone3": (-3 * FORMATION_SPACING, 0.0),
    "drone4": (-4 * FORMATION_SPACING, 0.0),
}

# V formation: two drones on each side.
V_OFFSETS = {
    "drone1": (-FORMATION_SPACING, FORMATION_SPACING),
    "drone2": (-FORMATION_SPACING, -FORMATION_SPACING),
    "drone3": (-2 * FORMATION_SPACING, 2 * FORMATION_SPACING),
    "drone4": (-2 * FORMATION_SPACING, -2 * FORMATION_SPACING),
}


# DRONE OPERATIONS
def initialize_drones():
    """Create the five AeroStack2 drone interfaces."""
    drones = {}

    for name in DRONE_NAMES:
        print(f"[INIT] Connecting to {name}...")
        drones[name] = DroneInterface(
            name,
            verbose=True,
            use_sim_time=True,
        )

    print("[INIT] All five interfaces initialized.")
    return drones
```

FIVE-DRONE SWARM

```
def takeoff_all(drones):
    """Arm, enter offboard mode, and take off."""
    print("\n[TAKEOFF] Preparing all five drones.")

    # Arm all drones first.
    for name in DRONE_NAMES:
        print(f"[ARM] {name}")
        drones[name].arm()

    # Enter offboard mode.
    for name in DRONE_NAMES:
        print(f"[OFFBOARD] {name}")
        drones[name].offboard()

    # Take off one at a time, waiting for each
    # blocking takeoff operation to complete.
    for name in DRONE_NAMES:
        print(f"[TAKEOFF] {name}")
        drones[name].takeoff(
            TAKEOFF_HEIGHT,
            TAKEOFF_SPEED,
            True,
        )

    print(f"[TAKEOFF] Waiting {STABILIZATION_TIME} seconds.")
    time.sleep(STABILIZATION_TIME)
    print("[TAKEOFF] Sequence completed.")


def move_to_formation(drones, formation_name):
    if formation_name == "LINE":
        offsets = LINE_OFFSETS
    elif formation_name == "V_FORMATION":
        offsets = V_OFFSETS
    else:
        raise ValueError(f"Unknown formation: {formation_name}")

    print(f"\n[LEADER COMMAND] {LEADER} selected {formation_name}")

    # Hold the leader at its nominal reference point.
    print("[LEADER] Moving to reference position.")
    drones[LEADER].go_to.go_to(
        LEADER_X,
        LEADER_Y,
        FLIGHT_HEIGHT,
        MISSION_SPEED,
        FRAME_ID,
    )

    # Command each follower to its formation position.
    for name in DRONE_NAMES:
        if name == LEADER:
            continue

        offset_x, offset_y = offsets[name]
        target_x = LEADER_X + offset_x
        target_y = LEADER_Y + offset_y

        print(
            f"[FOLLOWER] {name} -> "
            f"x={target_x:.2f}, "
            f"y={target_y:.2f}, "
            f"z={FLIGHT_HEIGHT:.2f}"
        )

        drones[name].go_to.go_to(
            target_x,
            target_y,
            FLIGHT_HEIGHT,
            MISSION_SPEED,
            FRAME_ID,
        )

    print(f"[FORMATION] {formation_name} command completed.")


def land_all(drones):
    """Request landing for all five drones."""
    print("\n[LAND] Starting landing sequence.")
```

```
    for name in DRONE_NAMES:
        print(f"[LAND] {name}")
        drones[name].land(LAND_SPEED, True)

    print("[LAND] Landing sequence completed.")


# COMMAND LOOP
def main():
    rclpy.init()

    drones = {}
    airborne = False

    print("=" * 55)
    print(" FIVE-DRONE LEADER-FOLLOWER FORMATION DEMO")
    print("=" * 55)
    print(f"Leader:    {LEADER}")
    print(f"Followers: {', '.join(DRONE_NAMES[1:])}")
    print()

    try:
        drones = initialize_drones()

        print("\nAvailable commands:")
        print("  TAKEOFF      - Take off all five drones")
        print("  LINE         - Switch to line formation")
        print("  V_FORMATION  - Switch to V formation")
        print("  LAND         - Land all five drones")
        print("  EXIT         - Exit the demo")
        print()

        while True:
            command = input("Leader command > ").strip().upper()

            if command == "TAKEOFF":
                if airborne:
                    print("[INFO] Drones are already marked airborne.")
                    continue
                takeoff_all(drones)
                airborne = True

            elif command in ("LINE", "V_FORMATION"):
                if not airborne:
                    print("[WARNING] Enter TAKEOFF first.")
                    continue
                move_to_formation(drones, command)

            elif command == "LAND":
                if not airborne:
                    print("[INFO] The takeoff sequence has not run.")
                    continue
                land_all(drones)
                airborne = False

            elif command == "EXIT":
                if airborne:
                    print(
                        "[SAFETY] Drones are marked airborne. "
                        "Use LAND before EXIT."
                    )
                    continue
                break

            elif command == "":
                continue

            else:
                print("[ERROR] Unknown command.")

    except KeyboardInterrupt:
        print(
            "\n[INTERRUPTED] Check the drones' actual state "
            "in Gazebo and land safely if necessary."
        )

    except Exception as exc:
        print(f"\n[ERROR] Demo stopped: {exc}")
```

```
            print(
                "[SAFETY] Check the actual state of every drone "
                "before issuing further commands."
            )
            raise

        finally:
            for name, drone in drones.items():
                try:
                    drone.shutdown()
                except Exception:
                    pass

            if rclpy.ok():
                rclpy.shutdown()


    if __name__ == "__main__":
        main()
```

Appendix 2: direct_formation.py

```
#!/usr/bin/env python3
import time, rclpy
from rclpy.node import Node
from rclpy.qos import qos_profile_sensor_data
from geometry_msgs.msg import Twist, PoseStamped

N = 5
LINE = [(0,0),(-1.5,0),(-3.0,0),(-4.5,0),(-6.0,0)]
V = [(0,0),(-1.5,1.5),(-1.5,-1.5),(-3.0,3.0),(-3.0,-3.0)]
ALT, GROUND = 1.05, 0.06
KP, VXY, VUP, VDN, TOL = 0.8, 0.6, 0.5, 0.4, 0.15


class Fleet(Node):
    def __init__(self):
        super().__init__('direct_formation')
        self.pos = [None]*N
        self.pub = []
        for i in range(N):
            self.create_subscription(
                PoseStamped, f'/drone{i}/ground_truth/pose',
                lambda m, i=i: self.pos.__setitem__(
                    i, (m.pose.position.x, m.pose.position.y, m.pose.position.z)),
                qos_profile_sensor_data)
            self.pub.append(self.create_publisher(Twist, f'/gz/drone{i}/cmd_vel', 10))
        while None in self.pos:
            rclpy.spin_once(self, timeout_sec=0.2)
        self.origin = self.pos[0][:2]

    @staticmethod
    def clip(v, lim): return max(-lim, min(lim, v))

    def goto(self, targets, vup=VUP, vdn=VDN, timeout=240, hold=2.0):
        ok_since, t0 = None, time.time()
        while time.time() - t0 < timeout:
            rclpy.spin_once(self, timeout_sec=0.05)
            worst = 0.0
            for i, (tx, ty, tz) in enumerate(targets):
                x, y, z = self.pos[i]
                ex, ey, ez = tx-x, ty-y, tz-z
                worst = max(worst, abs(ex), abs(ey), abs(ez))
                m = Twist()
                m.linear.x = self.clip(KP*ex, VXY)
                m.linear.y = self.clip(KP*ey, VXY)
                m.linear.z = self.clip(KP*ez*2, vup if ez > 0 else vdn)
                self.pub[i].publish(m)
            if worst < TOL:
                ok_since = ok_since or time.time()
                if time.time() - ok_since > hold: break
            else:
                ok_since = None
        print('  max error %.2f m' % worst)

    def formation(self, offs):
        ox, oy = self.origin
```

```
        self.goto([(ox+dx, oy+dy, ALT) for dx, dy in offs])

    def stop(self):
        for p in self.pub: p.publish(Twist())


def main():
    rclpy.init()
    f = Fleet()
    print('Commands: TAKEOFF LINE V_FORMATION LAND EXIT')
    while True:
        c = input('Leader command > ').strip().upper()
        if c == 'TAKEOFF':
            f.goto([(f.pos[i][0], f.pos[i][1], ALT) for i in range(N)])
        elif c == 'LINE': f.formation(LINE)
        elif c == 'V_FORMATION': f.formation(V)
        elif c == 'LAND':
            f.goto([(f.pos[i][0], f.pos[i][1], GROUND) for i in range(N)])
        elif c == 'EXIT': break
        f.stop()
    rclpy.shutdown()


main()
```

Appendix 3: leader_follower_dds.py

```
#!/usr/bin/env python3
import sys, time, threading, rclpy
from rclpy.node import Node
from rclpy.qos import qos_profile_sensor_data
from geometry_msgs.msg import Twist, PoseStamped
from std_msgs.msg import String

ALT, GROUND = 1.05, 0.06
KP, VXY, VUP, VDN = 0.8, 0.6, 0.5, 0.4
OFFSETS = {  # offsets from the leader (x, y), per follower id
    'LINE':  {1: (-1.5, 0), 2: (-3.0, 0), 3: (-4.5, 0), 4: (-6.0, 0)},
    'ARROW': {1: (-1.5, 1.5), 2: (-1.5, -1.5), 3: (-3.0, 3.0), 4: (-3.0, -3.0)},
}
KEYS = {'1': 'LINE', '2': 'ARROW', 'T': 'TAKEOFF', 'L': 'LAND',
        'LINE': 'LINE', 'ARROW': 'ARROW', 'TAKEOFF': 'TAKEOFF', 'LAND': 'LAND'}


def clip(v, lim): return max(-lim, min(lim, v))


class Base(Node):
    def __init__(self, idx, name):
        super().__init__(name)
        self.idx, self.pos = idx, None
        self.create_subscription(PoseStamped, f'/drone{idx}/ground_truth/pose',
                                 self._pose, qos_profile_sensor_data)
        self.pub = self.create_publisher(Twist, f'/gz/drone{idx}/cmd_vel', 10)

    def _pose(self, m):
        p = m.pose.position; self.pos = (p.x, p.y, p.z)

    def drive(self, t):
        x, y, z = self.pos
        ex, ey, ez = t[0]-x, t[1]-y, t[2]-z
        m = Twist()
        m.linear.x = clip(KP*ex, VXY); m.linear.y = clip(KP*ey, VXY)
        m.linear.z = clip(KP*ez*2, VUP if ez > 0 else VDN)
        self.pub.publish(m)
        return max(abs(ex), abs(ey), abs(ez))


class Leader(Base):
    def __init__(self):
        super().__init__(0, 'leader_drone0')
        self.cmd_pub = self.create_publisher(String, '/formation/command', 10)
        self.lp_pub = self.create_publisher(PoseStamped, '/formation/leader_pose', 10)
        self.target = None
        self.create_timer(0.05, self.loop)

    def loop(self):
```

```
        if self.pos is None: return
        m = PoseStamped(); m.header.frame_id = 'earth'
        m.pose.position.x, m.pose.position.y, m.pose.position.z = self.pos
        self.lp_pub.publish(m)
        if self.target: self.drive(self.target)

    def send(self, cmd):
        self.cmd_pub.publish(String(data=cmd))
        x, y, _ = self.target if self.target else self.pos
        if cmd == 'TAKEOFF': self.target = (self.pos[0], self.pos[1], ALT)
        if cmd == 'LAND': self.target = (x, y, GROUND)
        print(f'[LEADER] broadcast {cmd}')


class Follower(Base):
    def __init__(self, idx):
        super().__init__(idx, f'follower_drone{idx}')
        self.mode, self.fixed, self.leader, self.last = 'IDLE', None, None, 0
        self.create_subscription(String, '/formation/command', self.on_cmd, 10)
        self.create_subscription(PoseStamped, '/formation/leader_pose',
                                 self.on_leader, 10)
        self.create_timer(0.05, self.loop)

    def on_leader(self, m):
        p = m.pose.position; self.leader = (p.x, p.y, p.z)

    def on_cmd(self, m):
        cmd = m.data
        self.get_logger().info(f'received command from leader: {cmd}')
        if self.pos is None: return
        if cmd == 'TAKEOFF':
            self.mode, self.fixed = 'HOLD', (self.pos[0], self.pos[1], ALT)
        elif cmd == 'LAND':
            self.mode, self.fixed = 'HOLD', (self.pos[0], self.pos[1], GROUND)
        elif cmd in OFFSETS:
            self.mode = cmd

    def loop(self):
        if self.pos is None or self.mode == 'IDLE': return
        if self.mode == 'HOLD': t = self.fixed
        elif self.leader is None: return
        else:
            ox, oy = OFFSETS[self.mode][self.idx]
            t = (self.leader[0]+ox, self.leader[1]+oy, ALT)
        err = self.drive(t)
        if time.time() - self.last > 3:
            self.last = time.time()
            self.get_logger().info(f'mode={self.mode} error={err:.2f} m')


def main():
    rclpy.init()
    if sys.argv[1] == 'follower':
        rclpy.spin(Follower(int(sys.argv[2]))); return
    node = Leader()
    threading.Thread(target=rclpy.spin, args=(node,), daemon=True).start()
    while node.pos is None: time.sleep(0.2)
    print('Leader commands: 1=LINE 2=ARROW T=TAKEOFF L=LAND g dx dy=move leader q=quit')
    while True:
        c = input('Leader command > ').strip()
        if c.lower() == 'q': break
        if c.upper() in KEYS: node.send(KEYS[c.upper()])
        elif c.startswith('g '):
            try:
                dx, dy = map(float, c.split()[1:3])
            except ValueError:
                print('usage: g <dx> <dy>, e.g. g 2 0'); continue
            x, y, z = node.target or node.pos
            node.target = (x+dx, y+dy, z); print(f'[LEADER] moving by ({dx}, {dy})')
    rclpy.shutdown()


main()
```

Appendix 4: leader_tasks.py

```
#!/usr/bin/env python3
"""Leader assigns tasks, followers execute and report.
Usage: leader_tasks.py leader | follower <1-4>"""
import sys, time, json, threading, rclpy
from rclpy.node import Node
from rclpy.qos import qos_profile_sensor_data
from geometry_msgs.msg import Twist, PoseStamped
from std_msgs.msg import String

ALT, GROUND = 1.05, 0.06
KP, VXY, VUP, VDN, TOL = 0.8, 0.6, 0.5, 0.4, 0.2
OFFSETS = {
    'LINE':  {1: (-1.5, 0), 2: (-3.0, 0), 3: (-4.5, 0), 4: (-6.0, 0)},
    'ARROW': {1: (-1.5, 1.5), 2: (-1.5, -1.5), 3: (-3.0, 3.0), 4: (-3.0, -3.0)},
}
AREAS = {
    'A': [(-2.5, -2.0), (-2.5, 2.0), (-1.0, 2.0), (-1.0, -2.0)],
    'B': [(-0.5, -2.0), (-0.5, 2.0), (0.5, 2.0), (0.5, -2.0)],
    'C': [(1.0, -2.0), (1.0, 2.0), (2.5, 2.0), (2.5, -2.0)],
}
HELP = ('Leader commands: T=takeoff L=land 1=LINE 2=ARROW | mission [delay] | '
        'assign N HOLD|LAND|LINE|ARROW|GOTO x y z | g dx dy | status | q')


def clip(v, lim): return max(-lim, min(lim, v))
def alt_of(d): return ALT + 0.3 * (d - 1)   # separate altitude per drone
def wps(area, z): return [[x, y, z] for x, y in AREAS[area]]


class Base(Node):
    def __init__(self, idx, name):
        super().__init__(name)
        self.idx, self.pos = idx, None
        self.create_subscription(PoseStamped, f'/drone{idx}/ground_truth/pose',
                                 self._pose, qos_profile_sensor_data)
        self.pub = self.create_publisher(Twist, f'/gz/drone{idx}/cmd_vel', 10)

    def _pose(self, m):
        p = m.pose.position; self.pos = (p.x, p.y, p.z)

    def drive(self, t):
        x, y, z = self.pos
        ex, ey, ez = t[0]-x, t[1]-y, t[2]-z
        m = Twist()
        m.linear.x = clip(KP*ex, VXY); m.linear.y = clip(KP*ey, VXY)
        m.linear.z = clip(KP*ez*2, VUP if ez > 0 else VDN)
        self.pub.publish(m)
        return max(abs(ex), abs(ey), abs(ez))


class Leader(Base):
    def __init__(self):
        super().__init__(0, 'leader_drone0')
        self.lock = threading.Lock()
        self.lp_pub = self.create_publisher(PoseStamped, '/formation/leader_pose', 10)
        self.task_pub = {d: self.create_publisher(String, f'/formation/task/drone{d}', 10)
                         for d in (1, 2, 3, 4)}
        self.create_subscription(String, '/formation/status', self.on_status, 10)
        self.target, self.busy, self.done, self.owner = None, set(), set(), {}
        self.create_timer(0.05, self.loop)

    def loop(self):
        if self.pos is None: return
        m = PoseStamped(); m.header.frame_id = 'earth'
        m.pose.position.x, m.pose.position.y, m.pose.position.z = self.pos
        self.lp_pub.publish(m)
        if self.target: self.drive(self.target)

    def send_task(self, d, task):
        self.task_pub[d].publish(String(data=json.dumps(task)))
        print(f'[LEADER] -> drone{d}: {task["type"]}' +
              (f' area {task["area"]} waypoints {[i+1 for i in task["todo"]]}'
               if task['type'] == 'SURVEY' else ''))

    def broadcast(self, what):
        x, y, z = self.target or self.pos
        if what == 'TAKEOFF': self.target = (self.pos[0], self.pos[1], ALT)
```

FIVE-DRONE SWARM

```
        if what == 'LAND': self.target = (x, y, GROUND)
        task = {'type': 'FORMATION', 'name': what} if what in OFFSETS else {'type': what}
        with self.lock: self.busy.clear()
        for d in (1, 2, 3, 4): self.send_task(d, task)

    def start_mission(self, delay=False):
        with self.lock:
            self.done.clear(); self.owner = {}
        for d, area in ((1, 'A'), (2, 'B'), (3, 'C')):
            t = {'type': 'SURVEY', 'area': area, 'wps': wps(area, alt_of(d)),
                 'todo': [0, 1, 2, 3]}
            if delay and d == 1: t['delay_at'] = 2   # simulated delay at A3
            with self.lock:
                self.busy.add(d); self.owner[area] = d
            self.send_task(d, t)

    def assign_cmd(self, a):
        d, what = int(a[0]), a[1].upper()
        if what in ('HOLD', 'LAND'): t = {'type': what}
        elif what in OFFSETS: t = {'type': 'FORMATION', 'name': what}
        elif what == 'GOTO': t = {'type': 'GOTO', 'target': [float(v) for v in a[2:5]]}
        else: print('unknown task'); return
        with self.lock: self.busy.discard(d)
        self.send_task(d, t)

    def on_status(self, m):
        s = json.loads(m.data); d, ev = s['drone'], s['event']
        if ev == 'WP_DONE':
            with self.lock: self.done.add((s['area'], s['index']))
            print(f'[LEADER] drone{d} finished {s["area"]}{s["index"]+1}')
        elif ev == 'DELAYED':
            print(f'[LEADER] drone{d} reports DELAY, remaining {s["area"]} '
                  f'waypoints {[i+1 for i in s["remaining"]]}')
            self.reassign(d, s['area'], s['remaining'])
        elif ev == 'TASK_DONE':
            with self.lock: self.busy.discard(d)
            print(f'[LEADER] drone{d} finished its task')
            self.verify()
        elif ev == 'GOTO_DONE':
            print(f'[LEADER] drone{d} reached its GOTO point')

    def reassign(self, src, area, remaining):
        with self.lock:
            self.busy.discard(src)
            helpers = [d for d in (1, 2, 3, 4) if d not in self.busy and d != src]
            if not helpers:
                print('[LEADER] ERROR: no idle helper available'); return
            h = helpers[-1]   # drone4 is the spare
            self.busy.add(h)
        print(f'[LEADER] TRANSFER: drone{src} -> drone{h}, area {area}')
        self.send_task(h, {'type': 'SURVEY', 'area': area, 'wps': wps(area, alt_of(h)),
                           'todo': list(remaining)})
        self.send_task(src, {'type': 'HOLD'})

    def verify(self):
        with self.lock:
            ok = all((a, i) in self.done for a in AREAS for i in range(4))
            busy = bool(self.busy)
        if ok and not busy:
            print('[LEADER] MISSION VERIFIED: areas A, B, C 4/4 each, assigned by leader')

    def print_status(self):
        with self.lock:
            for a in AREAS:
                n = sum(1 for (ar, _) in self.done if ar == a)
                print(f'  Area {a}: {n}/4')
            print(f'  busy drones: {sorted(self.busy)}')


class Follower(Base):
    def __init__(self, idx):
        super().__init__(idx, f'follower_drone{idx}')
        self.task, self.leader = None, None
        self.st_pub = self.create_publisher(String, '/formation/status', 10)
        self.create_subscription(String, f'/formation/task/drone{idx}', self.on_task, 10)
        self.create_subscription(PoseStamped, '/formation/leader_pose', self.on_leader, 10)
        self.create_timer(0.05, self.loop)

    def status(self, event, **kw):
```

```
            kw.update(drone=self.idx, event=event)
            self.st_pub.publish(String(data=json.dumps(kw)))

        def on_leader(self, m):
            p = m.pose.position; self.leader = (p.x, p.y, p.z)

        def on_task(self, m):
            if self.pos is None: return
            d = json.loads(m.data); k = d['type']
            if k in ('HOLD', 'LAND', 'TAKEOFF'):
                x, y, z = self.pos
                d['target'] = [x, y, {'HOLD': z, 'LAND': GROUND, 'TAKEOFF': ALT}[k]]
            self.task = d
            self.get_logger().info(f'task from leader: {k} ' +
                                   str({a: d[a] for a in ('area', 'name', 'todo') if a in d}))

        def hold_here(self):
            self.task = {'type': 'HOLD', 'target': list(self.pos)}

        def loop(self):
            if self.pos is None or self.task is None: return
            t = self.task; k = t['type']
            if k in ('HOLD', 'LAND', 'TAKEOFF'):
                self.drive(t['target'])
            elif k == 'GOTO':
                if self.drive(t['target']) < TOL:
                    self.status('GOTO_DONE'); self.hold_here()
            elif k == 'FORMATION':
                if self.leader is None: return
                ox, oy = OFFSETS[t['name']][self.idx]
                self.drive((self.leader[0]+ox, self.leader[1]+oy, ALT))
            elif k == 'SURVEY':
                if not t['todo']:
                    self.status('TASK_DONE', area=t['area']); self.hold_here(); return
                i = t['todo'][0]
                if t.get('delay_at') == i and not t.get('delayed'):
                    t['delayed'] = True
                    self.status('DELAYED', area=t['area'], remaining=list(t['todo']))
                    self.hold_here(); return
                if self.drive(t['wps'][i]) < TOL:
                    self.status('WP_DONE', area=t['area'], index=i)
                    t['todo'].pop(0)


    def main():
        rclpy.init()
        if sys.argv[1] == 'follower':
            rclpy.spin(Follower(int(sys.argv[2]))); return
        L = Leader()
        threading.Thread(target=rclpy.spin, args=(L,), daemon=True).start()
        while L.pos is None: time.sleep(0.2)
        print(HELP)
        while True:
            try: c = input('Leader command > ').strip()
            except EOFError: break
            if not c: continue
            a = c.split(); k = a[0].upper()
            try:
                if k == 'Q': break
                elif k in ('T', 'L', '1', '2'):
                    L.broadcast({'T': 'TAKEOFF', 'L': 'LAND', '1': 'LINE', '2': 'ARROW'}[k])
                elif k == 'MISSION': L.start_mission(len(a) > 1 and a[1].lower() == 'delay')
                elif k == 'ASSIGN': L.assign_cmd(a[1:])
                elif k == 'G':
                    x, y, z = L.target or L.pos
                    L.target = (x+float(a[1]), y+float(a[2]), z)
                elif k == 'STATUS': L.print_status()
                else: print(HELP)
            except Exception as e:
                print('error:', e, '\n' + HELP)
        rclpy.shutdown()


    main()
```

Appendix 5: election.py

```
"""Dynamic leader election logic
Protocol (term-based, Raft-inspired, no voting rounds)
* A drone is ALIVE if its heartbeat was heard within `timeout` seconds.
* A claim is the pair (term, leader). Claims are totally ordered: higher
  term wins; for equal terms the LOWER leader id wins. Every drone adopts
  the highest claim it hears (whose leader it can see alive). This makes all
  views converge, even with packet loss.
* Initial election: after `startup` seconds with no claim heard, every drone
  independently computes the best-ranked alive drone and claims term 1.
* Failure: if my leader is not alive, I claim (term+1, best alive drone).
  Concurrent claims from different drones resolve via the ordering above.
* Planned handover: ONLY the current leader may start one. If a challenger
  beats it by the hysteresis margin, the leader claims (term+1, challenger).
  Followers never decide planned handovers on their own, so there is no
  threshold race and no flapping.
* Rejoin: a restarted drone has no claim, hears the swarm's claim and adopts
  it; it only becomes leader later through a planned handover.

Criteria
battery  : highest battery wins (tie -> lowest id); challenger must lead by
           `margin` percentage points to trigger a planned handover.
id       : lowest id wins; never pre-empts a live leader (bully-style).
centroid : drone closest to the swarm centroid wins; challenger must be
           closer by `centroid_margin` metres.
"""
import math


def assign_slots(leader, alive_ids):
    """Followers sorted by id get formation slots 1..n."""
    followers = sorted(i for i in alive_ids if i != leader)
    return {i: k + 1 for k, i in enumerate(followers)}


def claim_key(term, leader):
    return (term, -leader)


class Election:
    def __init__(self, my_id, criterion='battery', timeout=1.5, margin=5.0,
                 startup=3.0, t0=0.0, centroid_margin=1.0):
        if criterion not in ('battery', 'id', 'centroid'):
            raise ValueError('criterion must be battery, id or centroid')
        self.my_id, self.criterion = my_id, criterion
        self.timeout, self.margin = timeout, margin
        self.startup, self.t0 = startup, t0
        self.centroid_margin = centroid_margin
        self.peers = {}      # id -> {'battery','pos','seen','leader','term'}
        self.leader = None
        self.term = 0
        self.pinned = False   # True = manual leader, no automatic handovers

    # data in
    def update_self(self, battery, pos, now):
        self.peers[self.my_id] = {'battery': battery, 'pos': tuple(pos),
                                  'seen': now, 'leader': self.leader,
                                  'term': self.term}

    def hear(self, hb, now):
        if hb['id'] == self.my_id:
            return
        self.peers[hb['id']] = {'battery': hb['battery'],
                                'pos': tuple(hb['pos']), 'seen': now,
                                'leader': hb.get('leader'),
                                'term': hb.get('term', 0)}

    def alive_ids(self, now):
        return sorted(i for i, p in self.peers.items()
                      if now - p['seen'] <= self.timeout)

    # ranking
    def _dist(self, i, alive):
        xs = [self.peers[j]['pos'][0] for j in alive]
        ys = [self.peers[j]['pos'][1] for j in alive]
        cx, cy = sum(xs) / len(xs), sum(ys) / len(ys)
        x, y = self.peers[i]['pos'][:2]
        return math.hypot(x - cx, y - cy)
```

```
    def _key(self, i, alive):
        if self.criterion == 'battery':
            return (self.peers[i]['battery'], -i)
        if self.criterion == 'id':
            return (-i,)
        return (-self._dist(i, alive), -i)

    def best(self, alive):
        return max(alive, key=lambda i: self._key(i, alive))

    def _better_enough(self, challenger, current, alive):
        if self.criterion == 'battery':
            return (self.peers[challenger]['battery'] -
                    self.peers[current]['battery']) >= self.margin
        if self.criterion == 'id':
            return False
        return (self._dist(current, alive) -
                self._dist(challenger, alive)) >= self.centroid_margin

    # decision
    def _set(self, term, leader):
        self.term, self.leader = term, leader

    def force(self, target, now):
        """Manual handover (called on the CURRENT leader only). Returns True if done."""
        if target == self.leader or target not in self.alive_ids(now):
            return False
        self._set(self.term + 1, target)
        return True

    def elect(self, now):
        """Returns (leader, changed, reason)."""
        alive = self.alive_ids(now)
        if not alive:
            return self.leader, False, ''
        old = self.leader

        # 1. adopt the highest claim heard from a live peer naming a live leader
        claims = [(p['term'], p['leader']) for i, p in self.peers.items()
                  if i != self.my_id and i in alive
                  and p['leader'] is not None and p['leader'] in alive]
        if claims:
            top = max(claims, key=lambda c: claim_key(*c))
            mine = (claim_key(self.term, self.leader)
                    if self.leader is not None else (-1, 0))
            if claim_key(*top) > mine:
                self._set(*top)
                return self.leader, self.leader != old, \
                    f'adopted claim term {top[0]}'

        reason = ''
        if self.leader is None:
            if now - self.t0 >= self.startup:
                self._set(self.term + 1, self.best(alive))
                reason = 'initial election'
        elif self.leader not in alive:
            self._set(self.term + 1, self.best(alive))
            reason = f'leader drone{old} lost (heartbeat timeout)'
        elif self.leader == self.my_id and not self.pinned:
            b = self.best(alive)
            if b != self.my_id and self._better_enough(b, self.my_id, alive):
                self._set(self.term + 1, b)
                reason = f'{self.criterion} handover'
        return self.leader, self.leader != old, reason
```

Appendix 6: dynamic_swarm.py

```
#!/usr/bin/env python3
"""Dynamic leader election swarm: no hard-coded leader.

Every drone runs this SAME program:   dynamic_swarm.py <id 0-4> [--criterion battery|id|centroid]

Roles are decided at run time by election.py from heartbeats on /swarm/heartbeat.
The leader holds position; followers keep formation slots relative to it.
Operator commands arrive on /swarm/cmd (see swarm_cli.py).
"""
```

FIVE-DRONE SWARM

```
import sys, json, time, argparse, rclpy
from rclpy.node import Node
from rclpy.qos import qos_profile_sensor_data
from geometry_msgs.msg import Twist, PoseStamped
from std_msgs.msg import String
from election import Election, assign_slots

ALT, GROUND, ALT_STEP = 1.05, 0.06, 0.3
KP, VXY, VUP, VDN = 0.8, 0.6, 0.5, 0.4
OFFSETS = {   # (x, y) offset from the leader, by follower SLOT (1..4)
    'LINE':  {1: (-1.5, 0), 2: (-3.0, 0), 3: (-4.5, 0), 4: (-6.0, 0)},
    'ARROW': {1: (-1.5, 1.5), 2: (-1.5, -1.5), 3: (-3.0, 3.0), 4: (-3.0, -3.0)},
}
# Simulated battery (%). Replace with the real battery topic on hardware.
INIT_BATT = [100.0, 95.0, 90.0, 85.0, 80.0]
DRAIN, LEADER_EXTRA = 0.3, 0.3          # %/s airborne; leader works harder


def clip(v, lim): return max(-lim, min(lim, v))
def alt_of(i): return ALT + ALT_STEP * i   # one flight level per drone


class SwarmNode(Node):
    def __init__(self, idx, criterion):
        super().__init__(f'swarm_drone{idx}')
        self.idx, self.pos = idx, None
        self.battery, self.failed = INIT_BATT[idx], False
        self.mode, self.hold = 'IDLE', None
        self.el = Election(idx, criterion, t0=time.monotonic())
        self.last_t, self.last_log = time.monotonic(), 0.0
        self.create_subscription(PoseStamped, f'/drone{idx}/ground_truth/pose',
                                 self._pose, qos_profile_sensor_data)
        self.vel = self.create_publisher(Twist, f'/gz/drone{idx}/cmd_vel', 10)
        self.hb_pub = self.create_publisher(String, '/swarm/heartbeat', 10)
        self.create_subscription(String, '/swarm/heartbeat', self.on_hb, 10)
        self.create_subscription(String, '/swarm/cmd', self.on_cmd, 10)
        self.create_timer(0.05, self.loop)
        self.create_timer(0.10, self.beat)
        self.get_logger().info(f'drone{idx} up, criterion={criterion}, '
                               f'battery={self.battery:.0f}%')

    # inputs
    def _pose(self, m):
        p = m.pose.position; self.pos = (p.x, p.y, p.z)

    def on_hb(self, m):
        d = json.loads(m.data)
        if d['id'] != self.idx and not self.failed:
            self.el.hear(d, time.monotonic())

    def on_cmd(self, m):
        a = m.data.split()
        if not a or self.pos is None: return
        k = a[0].upper(); x, y, _ = self.pos
        try:
            if k in ('T', 'TAKEOFF'):
                self.mode, self.hold = 'HOLD', (x, y, alt_of(self.idx))
            elif k in ('L', 'LAND'):
                self.mode, self.hold = 'LANDING', (x, y, GROUND)
            elif k in ('1', 'LINE') and self.mode in ('HOLD', 'LINE', 'ARROW'):
                self.mode = 'LINE'
            elif k in ('2', 'ARROW') and self.mode in ('HOLD', 'LINE', 'ARROW'):
                self.mode = 'ARROW'
            elif k == 'G' and self.el.leader == self.idx and self.hold:
                hx, hy, hz = self.hold
                self.hold = (hx + float(a[1]), hy + float(a[2]), hz)
            elif k == 'FAIL' and int(a[1]) == self.idx:
                self.failed = True
                self.get_logger().warn(f'drone{self.idx}: SIMULATED FAILURE (silent, hovering)')
            elif k == 'RECOVER' and int(a[1]) == self.idx:
                self.failed = False
                pin = self.el.pinned
                self.el = Election(self.idx, self.el.criterion, t0=time.monotonic())
                self.el.pinned = pin
                self.get_logger().info(f'drone{self.idx}: recovered, rejoining swarm')
            elif k == 'BATT' and int(a[1]) == self.idx:
                self.battery = float(a[2])
            elif k == 'LEADER':
                n = int(a[1]); self.el.pinned = True
```

FIVE-DRONE SWARM

```
                if self.el.leader == self.idx:
                    if n == self.idx:
                        self.get_logger().info(f'MANUAL: drone{n} stays leader (pinned)')
                    elif self.el.force(n, time.monotonic()):
                        self.get_logger().info(f'MANUAL HANDOVER: drone{self.idx} -> drone{n} '
                                               f'(term {self.el.term}), pinned')
                    else:
                        self.get_logger().warn(f'MANUAL: drone{n} is not alive, request refused')
            elif k == 'AUTO':
                self.el.pinned = False
                if self.el.leader == self.idx:
                    self.get_logger().info('AUTO: automatic leader selection re-enabled')
        except (IndexError, ValueError):
            self.get_logger().warn(f'bad command: {m.data}')

    # outputs
    def beat(self):
        if self.failed or self.pos is None: return
        self.hb_pub.publish(String(data=json.dumps({
            'id': self.idx, 'battery': round(self.battery, 2),
            'pos': list(self.pos), 'mode': self.mode,
            'leader': self.el.leader, 'term': self.el.term,
            'pinned': self.el.pinned})))

    def drive(self, t):
        x, y, z = self.pos
        ex, ey, ez = t[0] - x, t[1] - y, t[2] - z
        m = Twist()
        m.linear.x = clip(KP * ex, VXY); m.linear.y = clip(KP * ey, VXY)
        m.linear.z = clip(KP * ez * 2, VUP if ez > 0 else VDN)
        self.vel.publish(m)
        return max(abs(ex), abs(ey), abs(ez))

    # main loop
    def loop(self):
        if self.pos is None: return
        now = time.monotonic(); dt, self.last_t = now - self.last_t, now
        if self.failed:                         # dead drone: hover, say nothing
            self.vel.publish(Twist()); return
        if self.mode != 'IDLE':
            rate = DRAIN + (LEADER_EXTRA if self.el.leader == self.idx else 0)
            self.battery = max(0.0, self.battery - rate * dt)
        self.el.update_self(self.battery, self.pos, now)
        old = self.el.leader
        leader, changed, reason = self.el.elect(now)
        if changed:
            self.get_logger().info(
                f'LEADER CHANGE: {old} -> drone{leader} (term {self.el.term}) [{reason}]')
            if leader == self.idx and self.mode in ('LINE', 'ARROW', 'HOLD'):
                self.hold = (self.pos[0], self.pos[1], alt_of(self.idx))
                self.get_logger().info(f'drone{self.idx} is now LEADER, holding position')
        if self.mode == 'IDLE' or leader is None: return
        if self.mode in ('HOLD', 'LANDING'):
            self.drive(self.hold); return
        if leader == self.idx:                  # LINE / ARROW leader
            self.drive(self.hold); return
        lp = self.el.peers.get(leader)
        if lp is None: return
        slots = assign_slots(leader, self.el.alive_ids(now))
        ox, oy = OFFSETS[self.mode][slots[self.idx]]
        err = self.drive((lp['pos'][0] + ox, lp['pos'][1] + oy, alt_of(self.idx)))
        if now - self.last_log > 3:
            self.last_log = now
            self.get_logger().info(f'{self.mode} slot {slots[self.idx]} behind drone{leader}, '
                                   f'err={err:.2f} m, battery={self.battery:.1f}%')


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument('id', type=int, choices=range(5))
    ap.add_argument('--criterion', default='battery', choices=['battery', 'id', 'centroid'])
    a = ap.parse_args()
    rclpy.init()
    rclpy.spin(SwarmNode(a.id, a.criterion))


if __name__ == '__main__':
```

```
    main()
```

Appendix 7: swarm_cli.py

```
#!/usr/bin/env python3
"""Operator console for the dynamic-leader swarm.

Commands:  T takeoff | L land | 1 LINE | 2 ARROW | g dx dy (moves current leader)
           leader N (manual leader) | auto (automatic again)
           fail N | recover N | batt N PERCENT | status | q
"""
import json, time, threading, rclpy
from rclpy.node import Node
from std_msgs.msg import String


class Cli(Node):
    def __init__(self):
        super().__init__('swarm_cli')
        self.pub = self.create_publisher(String, '/swarm/cmd', 10)
        self.hb = {}
        self.create_subscription(String, '/swarm/heartbeat', self.on_hb, 10)

    def on_hb(self, m):
        d = json.loads(m.data); d['t'] = time.monotonic(); self.hb[d['id']] = d

    def status(self):
        now = time.monotonic()
        print(' drone  state  battery  mode      sees leader  term')
        for i in range(5):
            d = self.hb.get(i)
            if d is None or now - d['t'] > 1.5:
                print(f'  {i}     DEAD/SILENT'); continue
            lead = 'LEADER ' if d['leader'] == i else 'follower'
            print(f'  {i}     {lead} {d["battery"]:6.1f}%  {d["mode"]:8s}  '
                  f'drone{d["leader"]}      {d["term"]}')
        views = {d['leader'] for d in self.hb.values() if now - d['t'] <= 1.5}
        print(' consensus:', 'YES' if len(views) == 1 else f'NO {views}')
        pins = {d.get('pinned') for d in self.hb.values() if now - d['t'] <= 1.5}
        print(' leader selection:', 'MANUAL (pinned)' if True in pins else 'AUTOMATIC')


def main():
    rclpy.init(); n = Cli()
    threading.Thread(target=rclpy.spin, args=(n,), daemon=True).start()
    print(__doc__)
    while True:
        try: c = input('swarm > ').strip()
        except EOFError: break
        if not c: continue
        if c.lower() == 'q': break
        if c.lower() == 'status': n.status()
        else: n.pub.publish(String(data=c))
    rclpy.shutdown()


if __name__ == '__main__':
    main()
```

Appendix 8: test_election.py

```
#!/usr/bin/env python3
"""Offline verification of the election logic: 5 virtual drones, simulated
10 Hz heartbeats with 20% random packet loss and a fake clock."""
import random
from election import Election, assign_slots

DT = 0.1


class Sim:
    def __init__(self, criterion='battery', batt=(100, 95, 90, 85, 80), loss=0.2,
                 drain=0.3, leader_extra=0.3, seed=1, positions=None):
        random.seed(seed)
        self.t = 0.0
```

FIVE-DRONE SWARM

```
        self.crit, self.loss = criterion, loss
        self.drain, self.extra = drain, leader_extra
        self.batt = {i: float(b) for i, b in enumerate(batt)}
        self.pos = positions or {i: (-1.5 * i, 0.0, 1.0) for i in range(5)}
        self.el = {i: Election(i, criterion) for i in range(5)}
        self.failed = set()
        self.changes = {i: [] for i in range(5)}   # (time, leader)
        self.airborne = False

    def step(self):
        self.t += DT
        for i, e in self.el.items():
            if i in self.failed:
                continue
            if self.airborne:
                rate = self.drain + (self.extra if e.leader == i else 0)
                self.batt[i] = max(0.0, self.batt[i] - rate * DT)
            e.update_self(self.batt[i], self.pos[i], self.t)
            ldr, ch, _ = e.elect(self.t)
            if ch:
                self.changes[i].append((round(self.t, 1), ldr))
        for i, e in self.el.items():          # broadcast
            if i in self.failed:
                continue
            hb = {'id': i, 'battery': self.batt[i], 'pos': self.pos[i],
                  'leader': e.leader, 'term': e.term}
            for j, o in self.el.items():
                if j != i and j not in self.failed and random.random() > self.loss:
                    o.hear(hb, self.t)

    def run(self, secs):
        for _ in range(int(secs / DT)):
            self.step()

    def views(self):
        return {i: e.leader for i, e in self.el.items() if i not in self.failed}

    def agreed(self):
        return len(set(self.views().values())) == 1


def test_initial_election():
    s = Sim(); s.run(6)
    assert s.agreed() and s.views()[0] == 0, s.views()
    print('PASS initial election -> drone0 (highest battery), all agree')


def test_leader_failure():
    s = Sim(); s.run(6); s.failed.add(0); s.run(5)
    assert s.agreed() and s.views()[1] == 1, s.views()
    t_fail = 6.0
    t_switch = max(t for i in (1, 2, 3, 4) for t, _ in s.changes[i][-1:])
    print(f'PASS leader failure -> drone1 elected by all, '
          f'recovery in {t_switch - t_fail:.1f} s')
    assert t_switch - t_fail < 3.0


def test_cascading_failure():
    s = Sim(); s.run(6); s.failed.add(0); s.run(5); s.failed.add(1); s.run(5)
    assert s.agreed() and s.views()[2] == 2, s.views()
    print('PASS cascading failure drone0 then drone1 -> drone2')


def test_rejoin_no_takeover():
    s = Sim(); s.run(6); s.failed.add(0); s.run(5)
    s.batt[0] = 92.0                                # weaker than drone1 (95)
    s.failed.discard(0)
    s.el[0] = Election(0, 'battery', t0=s.t)        # node restarts
    s.run(8)
    assert s.agreed() and s.views()[0] == 1, s.views()
    print('PASS weaker recovered drone0 adopts incumbent drone1, no forced re-election')


def test_rejoin_much_stronger_takes_over():
    s = Sim(); s.run(6); s.failed.add(0); s.run(5)
    s.failed.discard(0)
    s.el[0] = Election(0, 'battery', t0=s.t)        # rejoins with 100 vs 95
    s.run(8)
    assert s.agreed() and s.views()[0] == 0, s.views()
```

```
    print('PASS much stronger rejoiner regains leadership via planned handover')


def test_battery_handover_no_flapping():
    s = Sim(); s.run(6); s.airborne = True; s.run(300)
    ch = [(t, l) for t, l in s.changes[2]]
    assert s.agreed(), s.views()
    times = [t for t, _ in ch]
    dwell = [b - a for a, b in zip(times, times[1:])]
    assert len(ch) >= 2 and min(dwell) > 10, (ch, dwell)
    print(f'PASS battery handover: {len(ch)-1} planned handovers in 300 s, '
          f'min dwell {min(dwell):.0f} s (no flapping), all agree')


def test_equal_battery_stable():
    s = Sim(batt=(90, 90, 90, 90, 90)); s.airborne = False; s.run(60)
    assert s.agreed() and all(len(c) == 1 for c in s.changes.values())
    print('PASS equal battery: tie-break lowest id, zero flapping')


def test_id_and_centroid():
    s = Sim('id'); s.run(6); assert s.agreed() and s.views()[3] == 0
    s.failed.add(0); s.run(5); assert s.agreed() and s.views()[3] == 1
    pos = {0: (0, 0, 1), 1: (-10, 0, 1), 2: (-2, 0, 1), 3: (2, 0, 1), 4: (-1, 0, 1)}
    c = Sim('centroid', positions=pos); c.run(6)
    assert c.agreed() and c.views()[0] == 2, c.views()   # centroid x=-2.2 -> drone2
    print('PASS id criterion and centroid criterion')


def test_manual_leader_and_pin():
    s = Sim(); s.run(6)
    for e in s.el.values(): e.pinned = True                  # 'leader N' pins everyone first
    assert s.el[0].force(3, s.t)
    s.run(3); assert s.agreed() and s.views()[1] == 3, s.views()
    s.airborne = True; s.run(200)
    assert s.agreed() and s.views()[1] == 3, s.views()      # pinned: no auto handover
    s.failed.add(3); s.run(5)
    assert s.agreed() and 3 not in s.views().values()        # safety: failover still works
    ldr = s.views()[0]
    assert not s.el[ldr].force(3, s.t)                       # dead target refused
    for e in s.el.values(): e.pinned = False
    print('PASS manual leader: pin holds 200 s, failover still works, dead target refused')


def test_slots():
    assert assign_slots(1, [0, 1, 2, 3, 4]) == {0: 1, 2: 2, 3: 3, 4: 4}
    assert assign_slots(0, [0, 2, 3]) == {2: 1, 3: 2}
    print('PASS slot assignment with dead / changed leader')


if __name__ == '__main__':
    for f in (test_initial_election, test_leader_failure, test_cascading_failure,
              test_rejoin_no_takeover, test_rejoin_much_stronger_takes_over,
              test_battery_handover_no_flapping,
              test_equal_battery_stable, test_id_and_centroid, test_slots,
              test_manual_leader_and_pin):
        f()
    print('\nALL TESTS PASSED')
```

Appendix 9: run_leader_follower.sh

```
#!/bin/bash
source /opt/ros/humble/setup.bash
source ~/aerostack2_ws/install/setup.bash
export FASTRTPS_DEFAULT_PROFILES_FILE="$HOME/project_gazebo/config/fastdds_udp_only.xml"
export FASTDDS_DEFAULT_PROFILES_FILE="$HOME/project_gazebo/config/fastdds_udp_only.xml"
cd ~/project_gazebo
for i in 1 2 3 4; do python3 leader_follower_dds.py follower $i > /tmp/follower$i.log 2>&1 & done
trap 'kill $(jobs -p) 2>/dev/null' EXIT
sleep 3
python3 leader_follower_dds.py leader
```

Appendix 10: run_leader_tasks.sh

```
#!/bin/bash
source /opt/ros/humble/setup.bash
source ~/aerostack2_ws/install/setup.bash
export FASTRTPS_DEFAULT_PROFILES_FILE="$HOME/project_gazebo/config/fastdds_udp_only.xml"
export FASTDDS_DEFAULT_PROFILES_FILE="$HOME/project_gazebo/config/fastdds_udp_only.xml"
cd ~/project_gazebo
for i in 1 2 3 4; do python3 leader_tasks.py follower $i > /tmp/follower$i.log 2>&1 & done
trap 'kill $(jobs -p) 2>/dev/null' EXIT
sleep 3
python3 leader_tasks.py leader
```

Appendix 11: run_dynamic_swarm.sh

```
#!/bin/bash
# Usage: ./run_dynamic_swarm.sh [battery|id|centroid]
source /opt/ros/humble/setup.bash
source ~/aerostack2_ws/install/setup.bash
export FASTRTPS_DEFAULT_PROFILES_FILE="$HOME/project_gazebo/config/fastdds_udp_only.xml"
export FASTDDS_DEFAULT_PROFILES_FILE="$HOME/project_gazebo/config/fastdds_udp_only.xml"
cd ~/project_gazebo
CRIT="${1:-battery}"
for i in 0 1 2 3 4; do python3 dynamic_swarm.py $i --criterion "$CRIT" > /tmp/swarm$i.log 2>&1 & done
trap 'kill $(jobs -p) 2>/dev/null' EXIT
sleep 3
python3 swarm_cli.py
```